3. Python: Data Types

"You do not just code. You also need to know what types of data you are working with"

Previous Section:

</> 2. Python: Formatting

Contents:

In programming, data is everywhere. Numbers, words, logical values, and collections of information all need to be stored and processed correctly. But computers cannot treat every piece of data the same way. A number behaves differently from a sentence, and a list behaves differently from a single value. This is why data types exist.

Data types define what kind of value a variable contains and what operations can be performed on it. Understanding data types is extremely important because they affect calculations, comparisons, memory usage, and program behavior. A strong understanding of data types will help you write cleaner, safer, and more efficient Python programs.

1. Common Data Types in Python

Python data types are commonly divided into two categories: **Primitive data types and Non-primitive data types**

- Primitive data types are the most basic building blocks of data in Python. Some common primitive data types include:
  - `str` → Strings (text)
  - `int` → Integers (whole numbers)
  - `float` → Floating-point numbers (decimal numbers)
  - `bool` → Boolean values (`True` or `False`)
  - `None` → Represents the absence of a value

Examples:

```
print("Python")     # This is a string
print(3)            # This is an integer
print(3.12)         # This is a float
print(True, False)  # These are booleans
print(None)         # Represents no value
```

```
Python
3
3.12
True False
None
```

Notice this one:

```
print(3.12.0)
```

This is invalid because a number cannot contain multiple decimal points.

- Non-primitive data types are more complex structures that can store multiple values, often primitive data types grouped together. Some common non-primitive data types include: `list` , `tuple` , `set` , `dict`

Example:

```
print([1, 2, 3])                # List
print((1, 2, 3))                # Tuple
print({1, 2, 3})                # Set
print({"Name": "Alice"})        # Dictionary
```

```
[1, 2, 3]
(1, 2, 3)
{1, 2, 3}
{'Name': 'Alice'}
```

Python also allows us to convert between certain data types using built-in conversion functions: `str()` , `int()` , `float()` , `bool()`

Example:

```
num = 10

print(str(num))
print(float(num))
print(bool(num))
```

```
10
10.0
True
```

- The `str()` function is very flexible because almost everything in Python can be converted into a string.

```
print(str(100))
print(str(True))
print(str([1, 2, 3]))
```

```
100
True
[1, 2, 3]
```

- However, `int()` and `float()` are stricter because the value must actually represent a valid number.

When converting a float into an integer, Python removes the decimal portion instead of rounding.

```
num1 = 1.2
print(int(num1))

num2 = 1.5
print(int(num2))

num3 = 1.9
print(int(num3))
```

```
1
1
1
```

This behavior is different from `round()` because `int()` simply truncates the decimal part.

Converting strings into numbers only works if the string itself is numerical.

```
print(int('1'))
print(float('1.4'))
```

```
1
1.4
```

But these examples fail:

```
print(int('Python'))
print(int('1.4'))
```

Why?

- `'Python'` is not a number

- `'1.4'` contains a decimal point, so it cannot be directly converted into an integer using `int()`

Instead:

```
print(float('1.4'))
```

This one works correctly.

This becomes especially important for things like IDs. Even though IDs look numerical, they are often treated as strings instead of integers.

```
print("12330110")
```

12330110

This can be converted into an integer:

```
print(int("12330110"))
```

12330110

But this fails:

```
print(int("1234313x"))
```

Because the string contains a non-numerical character (`x`).

- Data type conversion is commonly used together with user input. Converting inputs into specific data types is good practice because it helps constrain and validate data properly inside programs.

```
name = str(input("Enter your name: "))
age = int(input("Enter your age: "))
print(name, age)
```

```
Enter your name: Alice
Enter your age: 12
Alice 12
```

Here, the program ensures that the user input for age becomes an integer instead of remaining as a string.

2. The `type()` Method

Sometimes, we may not know what data type a value belongs to. Python provides the `type()` function to help identify the type of an object.

This is extremely useful when debugging programs or checking whether a conversion worked correctly.

- Examples with primitive data types:

```
print(type('Hello'))
print(type(2))
print(type(True))
print(type(None))
print(type(2.5))
```

```
<class 'str'>
<class 'int'>
<class 'bool'>
<class 'NoneType'>
<class 'float'>
```

- The `type()` function also works with non-primitive data types.

```
print(type(['Hello', 'Goodbye']))
print(type((1, 2, 3)))
print(type({1, 2, 4, 5}))
print(type({'Name': 'Chris', 'Age': 30}))
```

```
<class 'list'>
<class 'tuple'>
<class 'set'>
<class 'dict'>
```

- Non-primitive data types can even contain other collections inside them.

```
print(type([[1, 2], [4, 5]]))
print(type([("Alice", 25), ("Bob", 30)]))
print(type({(1, 2), (2, 3), (3, 4)}))
```

```
<class 'list'>
<class 'list'>
<class 'set'>
```

This demonstrates an important concept in Python: Non-primitive data types are capable of storing primitive values and even other non-primitive structures together. This flexibility is one of the reasons Python is considered beginner-friendly and powerful at the same time.

3. Operations Between Data Types

One of the interesting things about Python is that different data types can sometimes interact with each other automatically. Python tries to make many operations flexible and beginner-friendly, but this flexibility also comes with rules that programmers need to understand carefully.

For example, two values may look similar even though they originally come from different data types.

```
print(int(12), int("12"))
```

```
12 12
```

Both outputs become integers, even though one started as a number and the other started as a string.

- Strings can also be combined together using the `+` operator.

```
print(str("12") + str("13.3"))
```

1213.3

Notice something important here: This is not mathematical addition. Python simply joins the two strings together. This process is called **string concatenation**.

- Mathematical operations only work correctly when the data types are compatible.

```
print(int(1) + int("1"))
print(float("3.13") + int(432))
```

2
435.13

Python automatically converts the values into numbers and performs arithmetic operations normally.

- A very special case in Python involves booleans.

```
# bool(True) returns 1
# bool(False) returns 0

print(bool(True) + bool(False))
print(bool(True) + bool(False) * 1000)
```

This works because internally:

- `True` behaves like `1`
- `False` behaves like `0`

So mathematically: `1 + 0 = 1`

However, not every operation between data types is allowed. The following examples will raise errors:

```
print(str("12") + int("13.3"))
print(str("12") + float("14.1"))
print(None + int(12))
print(None + str(14))
print(None + None)
```

Why do these fail? Because Python does not automatically combine incompatible types together. For example:

- Strings cannot directly add with integers or floats
- `None` represents the absence of a value, so mathematical operations involving `None` are undefined

A Special Catch About `bool()`

The `bool()` function may look simple at first, but it follows an important rule in Python:

- Empty or zero-like values become `False`
- Non-empty or non-zero values become `True`

Examples:

```python
string1 = " "
print(bool(string1))

string2 = 0
print(bool(string2))

string3 = 1
print(bool(string3))

string4 = -1
print(bool(string4))

string5 = None
print(bool(string5))

string6 = 1.0
print(bool(string6 + string6))
```

```
True
False
True
True
False
True
```

Results:

- `" "` → `True`
- `0` → `False`
- `1` → `True`
- `1` → `True`
- `None` → `False`
- `2.0` → `True`

This often surprises beginners. Why does `" "` become `True`? Because the string is not empty. Even though it only contains a space, it still has content.

But why are `0` and `None` considered `False`? Think about their meanings:

- `0` usually represents “nothing” in mathematics
- `None` literally represents “no value”

So Python treats them as logically false conditions.

This behavior becomes extremely important later in programming, especially in: `if` statements, loops, condition checking, input validation

Understanding how Python interprets truth values is one of the foundations of writing correct logical programs.

4. The `round()` Method

The `round()` function is used to round numbers in Python. It can either:

- Round a number to the nearest integer
- Round a number to a specific number of decimal places

This method only works with numerical data types such as `int` and `float`. It does not work directly with strings.

Basic examples:

```
x = round(5.1245, 2)
print(x)

y = round(5.773831883, 4)
print(y)

z = round(6.532)
print(z)
```

```
5.12
5.7738
7
```

Notice the second argument inside `round()`:

```
round(number, decimal_places)
```

This tells Python how many decimal places should remain after rounding. If no decimal place is provided, Python rounds the number to the nearest integer.

The Special Case

Now here comes one of the most interesting behaviors in Python rounding. If a number ends exactly in `.5`, Python does **not always round upward**.

Instead, Python uses a system called: **Round Half to Even** (also known as *Banker's Rounding*), This means Python rounds the number to the **nearest even integer**.

```
x1, y1 = 7.5, 5.5
x2, y2 = 6.5, 4.5

print(round(x1), round(y1))
print(round(x2), round(y2))
```

```
8 6
6 4
```

At first glance, this may look strange. Why does:

- `7.5` become `8` ; `5.5` become `6`
- but `6.5` becomes `6` and `4.5` becomes `4`

The reason is simple: Python checks the **nearest even number**.

Number	Rounded Result	Reason
7.5	8	8 is even
5.5	6	6 is even
6.5	6	6 is even
4.5	4	4 is even

This method helps reduce rounding bias in large calculations, especially in statistics and financial systems.

More Examples

```python
x1, y1 = 7.5, 5.5
x2, y2 = 6.5, 4.5

print(round(x1 + y1 + 0.5))
print(round(x2 - y2 + 0.5))
```

```
14
2
```

Again, Python chooses the nearest even integer.

This behavior surprises many beginners because most people expect `.5` to always round upward. But Python intentionally uses even rounding to make repeated calculations more balanced over time.

So while `round()` may seem like a simple function, it actually contains an important mathematical design choice behind the scenes.

Summary

Data types exist in every programming language because computers need to understand what kind of data they are working with. Python simply makes this process feel more natural and beginner-friendly.

In Python, we can write:

```python
a = 1
```

and Python automatically understands that `a` is an integer.

But in languages like C, programmers must explicitly define the type:

```c
int a = 1;
```

This feature is called **dynamic typing**, and it is one of the reasons Python is considered easier to learn.

In this section, we explored both primitive and non-primitive data types, learned how to convert between them, checked data types using the `type()` function, and examined how operations behave between different kinds of values.

We also discovered that Python has special logical rules for booleans, where values such as `0` and `None` are treated as `False`, while most non-empty and non-zero values become `True`.

Although Python hides many low-level details from the programmer, understanding data types is still extremely important. Every calculation, comparison, condition, and data structure in programming depends on them.

So while Python may feel simple on the surface, mastering data types is one of the key steps toward becoming a stronger programmer.

References

https://www.geeksforgeeks.org/python/python-data-types/

https://www.datacamp.com/blog/python-data-types

https://www.geeksforgeeks.org/python/python-type-function/

https://www.geeksforgeeks.org/python/round-function-python/

Next Section:

</> 4. Python: Conditional Statements